

K-Nearest Neighbors (KNN) Algorithm

Introduction to KNN

K-Nearest Neighbors (KNN) is a **supervised learning algorithm** used for **classification and regression** tasks. It's a non-parametric, instance-based algorithm that makes predictions based on the **k nearest data points** in the training set.

Non-Parametric

A parametric model (like linear regression) makes assumptions about the underlying data distribution and has a fixed number of parameters.

A non-parametric model like KNN does not make strong assumptions about the data distribution. Instead, it learns from the entire dataset dynamically, making it more flexible but also more computationally expensive.

Instance-Based

Instance-based learning means that KNN does not explicitly learn a model from the training data. Instead, it stores all the training data and makes predictions on the fly by comparing new inputs to stored instances.

This is different from model-based learning (e.g., neural networks), where the model generalizes and discards most of the training data after learning.

How KNN Works

1. Choose a value for k (the number of neighbors).
2. Measure the distance between the new data point and all existing points in the dataset (commonly using Euclidean distance).
3. Identify the k nearest neighbors (the 'k' closest points).
4. Classification → The new point is assigned the most common class among the k neighbors.
5. Regression → The prediction is the average of the k neighbors' values

Choosing the Right `k` Value

- A small `k` (e.g., 1 or 3) → More sensitive to noise, may lead to overfitting.
- A large `k` (e.g., 10, 20) → Leads to smoother decision boundaries but may underfit.
- A good rule of thumb is to choose `k = sqrt(n)`, where `n` is the number of data points.

Distance Metrics in KNN

KNN relies on measuring **distances** between points. Some common metrics:

1. **Euclidean Distance** (default for continuous data)

$$d((x, y), (a, b)) = \sqrt{(x - a)^2 + (y - b)^2}$$

2. **Manhattan Distance** (best for grid-like data)

$$L^1 = |x_2 - x_1| + |y_2 - y_1|$$

3. **Cosine Similarity** (for text data)

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

When to Use KNN?

Works well when

- The dataset is **small** to **medium-sized**.
- The data has **clear patterns** (low noise).
- The number of features is **not too large** (high-dimensional data may cause inefficiency).

Avoid when

- The dataset is **very large** (KNN is slow because it stores all training data).
- Data is **highly imbalanced**.
- There are **many irrelevant features** (can reduce accuracy).